

Module-7 Python – Collections , Functions and Modules

1. ACCESSING LIST

THEORY :

1. Understanding how to create and access elements in a list.



1. What is List?

A list is a collection of items stored in square brackets [].

Lists are ordered and changeable (mutable).

2. Creating a List

Example 1: Simple List

```
numbers = [10, 20, 30, 40]
```

Example 2: List with Different Data Types

```
data = [10, "Hello", 3.5, True]
```

Example 3: Empty List

```
my_list = []
```

3. Accessing List Elements

◆ Using Indexing

- Index starts from **0**

```
numbers = [10, 20, 30, 40]
```

```
print(numbers[0]) # 10
```

```
print(numbers[2]) # 30
```

◆ Negative Indexing

- Starts from the **end**

```
print(numbers[-1]) # 40
```

```
print(numbers[-2]) # 30
```

4. Accessing Multiple Elements (Slicing)

Syntax:

```
list[start:end]
```

Example:

```
numbers = [10, 20, 30, 40, 50]
```

```
print(numbers[1:4]) # [20, 30, 40]
```

5. Changing List Elements

(List is mutable )

```
numbers = [10, 20, 30]
```

```
numbers[1] = 99
```

```
print(numbers) # [10, 99, 30]
```

6. Looping Through a List

```
numbers = [10, 20, 30]
```

```
for num in numbers:
```

```
    print(num)
```

2. Indexing in lists (positive and negative indexing).



Indexing is used to **access elements** of a list using their position.

1. Positive Indexing

- Starts from **0**
- Moves from **left to right**

```
numbers = [10, 20, 30, 40]
```

```
print(numbers[0]) # 10
```

```
print(numbers[2]) # 30
```

2. Negative Indexing

- Starts from **-1** (end)
- Moves from **right to left**

```
print(numbers[-1]) # 40
```

```
print(numbers[-2]) # 30
```

3. Slicing a list: accessing a range of elements.

➔ Slicing is used to access a part (range) of a list.

It returns a new list with selected elements.

2. Syntax

```
list[start : end]
```

3. Example

```
numbers = [10, 20, 30, 40, 50]
```

```
print(numbers[1:4])
```

Output:

```
[20, 30, 40]
```

4. Different ways of Slicing

1. From Start to End

```
print(numbers[0:3]) # [10, 20, 30]
```

2. Without Start Index

```
print(numbers[:3]) # [10, 20, 30]
```

3. Without End Index

```
print(numbers[2:]) # [30, 40, 50]
```

4. Full List Copy

```
print(numbers[:]) # [10, 20, 30, 40, 50]
```

5. Negative Index Slicing

```
print(numbers[-4:-1]) # [20, 30, 40]
```

6. Slicing with Step

```
list[start : end : step]
```

```
print(numbers[0:5:2]) # [10, 30, 50]
```

2. LIST OPERATIONS

THEORY :

1. Common list operations: concatenation, repetition, membership.



1. Concatenation (+)

- Used to **join two or more lists**
- Creates a **new list**

Example:

```
list1 = [1, 2, 3]
```

```
list2 = [4, 5, 6]
```

```
result = list1 + list2
```

```
print(result)
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

2. Repetition (*)

- Used to **repeat a list multiple times**

Example:

```
numbers = [1, 2, 3]
```

```
print(numbers * 3)
```

Output:

[1, 2, 3, 1, 2, 3, 1, 2, 3]

3. Membership (in / not in)

- Used to check whether an element **exists in a list**
- Returns **True or False**

Example:

```
numbers = [10, 20, 30, 40]
```

```
print(20 in numbers)    # True
```

```
print(50 in numbers)    # False
```

```
print(50 not in numbers) # True
```

2. Understanding list methods like `append()`, `insert()`, `remove()`, `pop()`.



1. `append()`

- Adds an element at the **end of the list**

Example:

```
numbers = [1, 2, 3]
```

```
numbers.append(4)  
print(numbers)
```

Output:

```
[1, 2, 3, 4]
```

2. `insert()`

- Adds an element at a **specific position**

Syntax:

```
list.insert(index, element)
```

Example:

```
numbers = [1, 2, 3]
```

```
numbers.insert(1, 10)  
print(numbers)
```

Output:

```
[1, 10, 2, 3]
```

3. remove()

- Removes a **specific value**
- Deletes the **first occurrence**

Example:

```
numbers = [1, 2, 3, 2]
```

```
numbers.remove(2)  
print(numbers)
```

Output:

```
[1, 3, 2]
```

4. pop()

- Removes an element using its **index**
- Returns the removed value

Example:

```
numbers = [1, 2, 3]
```

```
numbers.pop(1)  
print(numbers)
```

Output:

```
[1, 3]
```

Default Behavior:

```
numbers.pop()
```

Removes **last element**

3. WORKING WITH LISTS

THEORY :

1. Iterating over a list using loops.



Iteration means **traversing (going through) each element** of a list one by one.

1. Using for Loop (Most Common)

Example:

```
numbers = [10, 20, 30, 40]
```

```
for num in numbers:  
    print(num)
```

Output:

```
10  
20  
30  
40
```

2. Using for Loop with Index (range)

Example:

```
numbers = [10, 20, 30, 40]
```

```
for i in range(len(numbers)):  
    print(numbers[i])
```

Output:

10
20
30
40

3. Using while Loop**Example:**

```
numbers = [10, 20, 30, 40]
```

```
i = 0
```

```
while i < len(numbers):
```

```
    print(numbers[i])
```

```
    i += 1
```

2. Sorting and reversing a list using sort(), sorted(), and reverse().



1. sort() Method

- Sorts the list in **ascending order**
- **Changes the original list (in-place)**

Example:

```
numbers = [5, 2, 9, 1]
```

```
numbers.sort()  
print(numbers)
```

Output:

```
[1, 2, 5, 9]
```

◆ Descending Order:

```
numbers.sort(reverse=True)  
print(numbers)
```

Output:

```
[9, 5, 2, 1]
```

2. sorted() Function

- Returns a **new sorted list**
- Does **not change original list**

Example:

```
numbers = [5, 2, 9, 1]
```

```
new_list = sorted(numbers)
```

```
print(new_list)
print(numbers)
```

Output:

```
[1, 2, 5, 9]
[5, 2, 9, 1]
```

◆ **Descending Order:**

```
new_list = sorted(numbers, reverse=True)
```

3. reverse() Method

- Reverses the list order
- **Does not sort**, only flips order
- Changes original list

Example:

```
numbers = [1, 2, 3, 4]
```

```
numbers.reverse()
print(numbers)
```

Output:

```
[4, 3, 2, 1]
```

3. Basic list manipulations: addition, deletion, updating, and slicing.



1. Addition (Adding Elements)

◆ Using `append()` (add at end)

```
numbers = [1, 2, 3]
numbers.append(4)
print(numbers)
```

Output: [1, 2, 3, 4]

◆ Using `insert()` (add at specific index)

```
numbers = [1, 2, 3]
numbers.insert(1, 10)
print(numbers)
```

Output: [1, 10, 2, 3]

◆ Using `+` (concatenation)

```
a = [1, 2]
b = [3, 4]
print(a + b)
```

Output: [1, 2, 3, 4]

2. Deletion (Removing Elements)

◆ Using `remove()` (by value)

```
numbers = [1, 2, 3]
numbers.remove(2)
print(numbers)
```

Output: [1, 3]

◆ Using pop() (by index)

```
numbers = [1, 2, 3]
numbers.pop(1)
print(numbers)
```

Output: [1, 3]

◆ Using del (delete by index or full list)

```
numbers = [1, 2, 3]
del numbers[0]
print(numbers)
```

Output: [2, 3]

3. Updating Elements

- Lists are **mutable**, so values can be changed

```
numbers = [1, 2, 3]
numbers[1] = 99
print(numbers)
```

Output: [1, 99, 3]

4. Slicing (Accessing Range)

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4])
```

Output: [20, 30, 40]

4. TUPLE

THEORY :

1. Introduction to tuples, immutability.



1. What is a Tuple?

- A **tuple** is a collection of elements.
- It is **ordered** and **immutable (cannot be changed)**.
- Written using **parentheses ()**

2. Creating a Tuple

Example:

```
t = (10, 20, 30)
print(t)
```

Different Types:

```
t1 = (1, 2, 3)          # integers
t2 = ("a", "b", "c")    # strings
t3 = (1, "Hello", 3.5)  # mixed
```

3. Accessing Elements

(Same as lists using indexing)

```
t = (10, 20, 30)

print(t[0]) # 10
print(t[-1]) # 30
```

4. What is Immutability?

- Immutability means the data cannot be changed after creation.

✗ Example (Error):

```
t = (10, 20, 30)
t[1] = 50 # Error
```

2. Creating and accessing elements in a tuple.



1. Creating a Tuple

- A tuple is created using **parentheses ()**

Example:

```
t = (10, 20, 30)
print(t)
```

Different Types:

```
t1 = (1, 2, 3)          # integers
t2 = ("a", "b", "c")    # strings
t3 = (1, "Hello", 3.5)  # mixed
```

2. Accessing Tuple Elements

◆ Using Positive Indexing

- Starts from **0**

```
t = (10, 20, 30)

print(t[0]) # 10
print(t[2]) # 30
```


◆ Using Negative Indexing

- Starts from -1

```
print(t[-1]) # 30
```

```
print(t[-2]) # 20
```

3. Accessing Multiple Elements (Slicing)

Syntax:

```
tuple[start:end]
```

Example:

```
t = (10, 20, 30, 40, 50)
```

```
print(t[1:4])
```

Output:

```
(20, 30, 40)
```

4. Looping Through a Tuple

```
t = (10, 20, 30)
```

```
for item in t:
```

```
    print(item)
```

3. Basic operations with tuples: concatenation, repetition, membership.

→

1. Concatenation (+)

- Used to **join two tuples**
- Creates a **new tuple**

Example:

```
t1 = (1, 2, 3)
```

```
t2 = (4, 5, 6)
```

```
result = t1 + t2
```

```
print(result)
```

Output:

```
(1, 2, 3, 4, 5, 6)
```

2. Repetition (*)

- Used to **repeat a tuple multiple times**

Example:

```
t = (1, 2)
```

```
print(t * 3)
```

Output:

```
(1, 2, 1, 2, 1, 2)
```

3. Membership (in / not in)

- Used to check whether an element **exists in a tuple**
- Returns **True or False**

Example:

```
t = (10, 20, 30)
```

```
print(20 in t)    # True
```

```
print(50 in t)    # False
```

```
print(50 not in t) # True
```

5. ACCESSING TUPLES

THEORY :

1. Accessing tuple elements using positive and negative indexing.



◆ Using Positive Indexing

- Starts from **0**

```
t = (10, 20, 30)
```

```
print(t[0]) # 10
```

```
print(t[2]) # 30
```

◆ Using Negative Indexing

- Starts from **-1**

```
print(t[-1]) # 30
```

```
print(t[-2]) # 20
```

2. Slicing a tuple to access ranges of elements.



- **Slicing** is used to **access a part (range) of a tuple**.
- It returns a **new tuple** with selected elements.
- Tuples are **immutable**, but slicing is allowed.

2. Syntax

```
tuple[start : end]
```

- **start** → starting index
- **end** → ending index (**not included**)

3. Example

```
t = (10, 20, 30, 40, 50)
```

```
print(t[1:4])
```

Output:

```
(20, 30, 40)
```

4. Different Slicing Cases

◆ 1. From Start to Given Index

```
print(t[:3]) # (10, 20, 30)
```

◆ 2. From Index to End

```
print(t[2:]) # (30, 40, 50)
```

◆ 3. Full Tuple Copy

```
print(t[:]) # (10, 20, 30, 40, 50)
```

◆ 4. Negative Index Slicing

```
print(t[-4:-1]) # (20, 30, 40)
```

5. Slicing with Step

```
tuple[start : end : step]
```

Example:

```
print(t[0:5:2]) # (10, 30, 50)
```

👉 Step = 2 → skips elements

6. DICTIONARIES

THEORY :

1. Introduction to dictionaries: key-value pairs.



1. What is a Dictionary?

- A **dictionary** is a collection of data stored in **key–value pairs**.
- It is **unordered, mutable, and indexed by keys**.
- Written using **curly braces { }**

2. Key–Value Pair Concept

- Each element has:
 - **Key** → unique identifier
 - **Value** → data stored

Example:

```
student = {  
    "name": "John",  
    "age": 20,  
    "marks": 85  
}
```

- "name" → key
- "John" → value

3. Characteristics of Dictionary

- Keys must be **unique**
- Values can be **duplicate**
- Keys must be **immutable** (string, number, tuple)
- Dictionary is **mutable** (can change values)

4. Accessing Values

```
print(student["name"]) # John  
print(student["age"]) # 20
```

5. Adding or Updating Values

```
student["age"] = 21      # update  
student["city"] = "Delhi" # add new
```

6. Deleting Elements

```
del student["marks"]
```

7. Example Output

```
print(student)
```

2. Accessing, adding, updating, and deleting dictionary elements.



1. Accessing Elements

- Use the **key** to access its value

Example:

```
student = {  
    "name": "Ali",  
    "age": 20,  
    "marks": 85  
}
```

```
print(student["name"]) # Ali  
print(student["age"]) # 20
```

2. Adding Elements

- Add a new key–value pair

Example:

```
student["city"] = "Delhi"  
print(student)
```

Output:

```
{'name': 'Ali', 'age': 20, 'marks': 85, 'city': 'Delhi'}
```

3. Updating Elements

- Modify the value of an existing key

Example:

```
student["age"] = 21  
print(student)
```

4. Deleting Elements

◆ Using del

```
del student["marks"]  
print(student)
```

◆ Using pop()

```
student.pop("age")  
print(student)
```

◆ Using clear() (remove all elements)

```
student.clear()  
print(student)
```

3. Dictionary methods like keys(), values(), and items().



1. keys() Method

- Returns all the **keys** in the dictionary

Example:

```
student = {  
    "name": "John",  
    "age": 20,  
    "marks": 85  
}
```

```
print(student.keys())
```

Output:

```
dict_keys(['name', 'age', 'marks'])
```

2. values() Method

- Returns all the **values** in the dictionary

Example:

```
print(student.values())
```

Output:

```
dict_values(['John', 20, 85])
```

3. items() Method

- Returns all **key–value pairs** as tuples

Example:

```
print(student.items())
```

Output:

```
dict_items([('name', 'John'), ('age', 20), ('marks', 85)])
```


7. WORKING WITH DICTIONARIES

THEORY :

1. Iterating over a dictionary using loops.

➔ Iteration means **traversing each element (key/value)** in a dictionary.

1. Looping Through Keys (Default)

- By default, a loop goes through keys

Example:

```
student = {  
    "name": "John",  
    "age": 20,  
    "marks": 85  
}
```

```
for key in student:  
    print(key)
```

Output:

```
name  
age  
marks
```

2. Looping Through Values

Example:

```
for value in student.values():  
    print(value)
```

Output:

```
John  
20  
85
```

3. Looping Through Keys and Values

Using items() (Most Important ★)

```
for key, value in student.items():  
    print(key, value)
```

Output:

```
name John  
age 20  
marks 85
```

2. Merging two lists into a dictionary using loops or zip().



1. Using zip()

- zip() combines elements of two lists into pairs

Example:

```
keys = ["name", "age", "marks"]  
values = ["John", 20, 85]
```

```
result = dict(zip(keys, values))  
print(result)
```

Output:

```
{'name': 'John', 'age': 20, 'marks': 85}
```

2. Using Loop

- Create dictionary manually using loop

Example:

```
keys = ["name", "age", "marks"]
```

```
values = ["John", 20, 85]
```

```
result = {}
```

```
for i in range(len(keys)):
```

```
    result[keys[i]] = values[i]
```

```
print(result)
```

Output:

```
{'name': 'John', 'age': 20, 'marks': 85}
```

3. Counting occurrences of characters in a string using dictionaries.

```
text = "hello"
```

```
count = {}
```

```
for ch in text:
```

```
    if ch in count:
```

```
        count[ch] += 1
```

```
    else:
```

```
        count[ch] = 1
```

```
print(count)
```

Output:

```
{'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

8. FUNCTIONS

THEORY :

1. Defining functions in Python.



1. What is a Function?

- A **function** is a block of code used to perform a specific task.
- It helps in **code reuse** and better organization.

2. Syntax of Function

```
def function_name(parameters):  
    statement(s)
```

3. Example

```
def greet():  
    print("Hello, World!")
```

```
greet()
```

Output:

Hello, World!

4. Function with Parameters

```
def greet(name):  
    print("Hello", name)
```

```
greet("John")
```

Output:

Hello John

5. Function with Return Value

```
def add(a, b):  
    return a + b
```

```
result = add(5, 3)  
print(result)
```

Output:

8

6. Types of Functions

◆ 1. Built-in Functions

- Already available in Python
- Example: print(), len(), input(), type()

◆ 2. User-defined Functions

- Created by programmer using def

2. Different types of functions: with/without parameters, with/without return values.



1. Function WITHOUT Parameters & WITHOUT Return Value

- Takes no input
- Returns nothing

Example:

```
def greet():  
    print("Hello, John")
```

```
greet()
```

 Output:

Hello, John

2. Function WITH Parameters & WITHOUT Return Value

- Takes input
- Does not return anything

Example:

```
def greet(name):  
    print("Hello", name)
```

```
greet("John")
```

Output:

Hello John

3. Function WITHOUT Parameters & WITH Return Value

- Takes no input
- Returns a value

Example:

```
def get_number():  
    return 10
```

```
x = get_number()  
print(x)
```

Output:

10

4. Function WITH Parameters & WITH Return Value

- Takes input
- Returns result

Example:

```
def add(a, b):  
    return a + b
```

```
result = add(5, 3)  
print(result)
```

Output:

8

3. Anonymous functions (lambda functions).



1. What is a Lambda Function?

- A **lambda function** is a **small anonymous function** (function without a name).
- It is defined using the keyword **lambda**.
- Used for **short, simple operations**.

2. Syntax

lambda arguments : expression

3. Example (Simple Lambda Function)

```
add = lambda a, b: a + b  
print(add(5, 3))
```

Output:

8

4. Example Without Assigning to Variable

```
print((lambda x: x * 2)(5))
```

✓ **Output:**

10

5. Using Lambda with Built-in Functions

◆ Using with map()

```
numbers = [1, 2, 3]
```

```
result = list(map(lambda x: x * 2, numbers))  
print(result)
```

✓ **Output:**

[2, 4, 6]

◆ Using with filter()

```
numbers = [1, 2, 3, 4]
```

```
result = list(filter(lambda x: x % 2 == 0, numbers))  
print(result)
```

✓ **Output:**

[2, 4]

9. MODULES

THEORY :

1. Introduction to Python modules and importing modules.



1. What is a Module?

- A **module** is a file containing Python code (functions, variables, classes).
- It helps in **organizing code** and **reusing functionality**.

2. Why Use Modules?

- Code reuse
- Better organization
- Avoid repetition
- Easy maintenance

3. Types of Modules

◆ 1. Built-in Modules

- Already available in Python
- Examples:
 - math
 - random
 - datetime

◆ 2. User-defined Modules

- Created by the user
- Example:

```
# file: mymodule.py
def greet():
    print("Hello, John")
```

Importing Modules

1. Import Entire Module

```
import math

print(math.sqrt(16))
```

2. Import Specific Function

```
from math import sqrt

print(sqrt(25))
```

3. Import with Alias

```
import math as m

print(m.sqrt(36))
```

4. Import All Functions

```
from math import *

print(sqrt(49))
```

5. Using User-defined Module

```
import mymodule

mymodule.greet()
```

2. Standard library modules: math, random.



Python provides many **built-in modules** called the **standard library**.

Two important modules are:

- math → for mathematical operations
- random → for generating random values

1. math Module

- Used for **mathematical calculations**
- Must be imported before use

Example:

```
import math
```

```
print(math.sqrt(16)) # 4.0  
print(math.pow(2, 3)) # 8.0
```

Common Functions in math Module

Function	Description
math.sqrt(x)	Square root
math.pow(x, y)	Power
math.ceil(x)	Round up
math.floor(x)	Round down

2. random Module

- Used to generate **random numbers or choices**

Example:

```
import random
```

```
print(random.random()) # random float (0 to 1)
```

```
print(random.randint(1, 10)) # random integer
```

Common Functions in random Module

Function	Description
random.random()	Float between 0 and 1
random.randint(a, b)	Integer between a and b
random.choice(list)	Random element
random.shuffle(list)	Shuffle list

3. Creating custom modules.



1. What is a Custom Module?

- A **custom module** is a Python file (.py) created by the user.
- It contains **functions, variables, or classes**.
- Used to **reuse code in different programs**.

2. Steps to Create a Module

◆ Step 1: Create a Python File

Create a file named mymodule.py

```
# mymodule.py
def greet():
    print("Hello, John")

def add(a, b):
    return a + b
```

◆ Step 2: Import the Module in Another File

```
import mymodule

mymodule.greet()
print(mymodule.add(5, 3))
```

Output:

```
Hello, John
8
```

3. Different Ways to Import Custom Module

◆ 1. Import Full Module

```
import mymodule
```

◆ 2. Import Specific Function

```
from mymodule import greet
```

```
greet()
```

◆ 3. Import with Alias

```
import mymodule as m
```

```
m.greet()
```

Module-6 Python – Fundamentals of Python Language

6. GENERATORS AND ITERATORS

THEORY :

1. Understanding how generators work in Python.



1. What is a Generator?

- A **generator** is a special type of function that **returns values one at a time** using the keyword **yield**.
- It does **not store all values in memory** at once.

2. Syntax of Generator

```
def function_name():  
    yield value
```

3. Example of Generator

```
def numbers():  
    for i in range(5):  
        yield i
```

```
for num in numbers():  
    print(num)
```

Output:

```
0  
1  
2  
3  
4
```

4. How Generator Works

1. Function is called → returns a **generator object**
2. Execution pauses at yield
3. Next value is produced when needed
4. Continues until function ends

5. Using next() with Generator

```
def count():
```

```
    yield 1
```

```
    yield 2
```

```
    yield 3
```

```
g = count()
```

```
print(next(g)) # 1
```

```
print(next(g)) # 2
```

```
print(next(g)) # 3
```

2. Difference between yield and return.



1. Basic Meaning

- **return** → Ends the function and sends back a value
- **yield** → Produces a value and **pauses** the function (used in generators)

2. Key Differences

Feature	return	yield
Function Type	Normal function	Generator function
Execution	Ends function	Pauses function

Feature	return	yield
Values	Returns one value	Returns multiple values (one by one)
Memory	Uses more memory	Memory efficient
Resume	Cannot resume	Can resume from last point

3. Example Using return

```
def demo():  
    return 1  
    return 2  
  
print(demo())
```

Output:

1

👉 Function stops after first return

4. Example Using yield

```
def demo():  
    yield 1  
    yield 2  
  
for i in demo():  
    print(i)
```

Output:

1

2

👉 Function continues after each yield

3. Understanding iterators and creating custom iterators.



1. What is an Iterator?

- An **iterator** is an object that allows you to **traverse (loop through) elements one by one**.
- It implements two methods:
 - `__iter__()`
 - `__next__()`

2. How Iterators Work

- `__iter__()` → returns the iterator object
- `__next__()` → returns next value
- When no values left → raises **StopIteration**

3. Example of Iterator

```
numbers = [1, 2, 3]
```

```
it = iter(numbers)
```

```
print(next(it)) # 1
```

```
print(next(it)) # 2
```

```
print(next(it)) # 3
```

Creating Custom Iterator

Example: Print numbers from 1 to 3

```
class MyNumbers:
    def __init__(self):
        self.num = 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.num <= 3:
            x = self.num
            self.num += 1
            return x
        else:
            raise StopIteration
```

Using the Iterator

```
obj = MyNumbers()
```

```
for i in obj:
    print(i)
```

Output:

```
1
2
3
```

7. FUNCTIONS AND METHODS

THEORY :

1. Defining and calling functions in Python.



1. Defining a Function

- A function is defined using the keyword **def**

Syntax:

```
def function_name(parameters):  
    statement(s)
```

2. Example (Defining a Function)

```
def greet():  
    print("Hello, John")
```

👉 Here:

- greet is the function name
- No parameters
- It prints a message

3. Calling a Function

- To execute a function, we **call it using its name**

```
greet()
```

Output:

Hello, John

4. Function with Parameters

```
def greet(name):  
    print("Hello", name)
```

```
greet("John")
```

Output:

Hello John

5. Function with Return Value

```
def add(a, b):  
    return a + b
```

```
result = add(5, 3)  
print(result)
```

Output:

8

2. Function arguments (positional, keyword, default).



Function Arguments in Python

Function arguments are values passed to a function when it is called.

1. Positional Arguments

- Arguments are passed **in the same order** as parameters

Example:

```
def greet(name, age):  
    print(name, age)
```

```
greet("John", 20)
```

Output:

John 20

👉 Order is important:

```
greet(20, "John") # Wrong meaning
```

2. Keyword Arguments

- Arguments are passed using **parameter names**
- Order does **not matter**

Example:

```
def greet(name, age):  
    print(name, age)
```

```
greet(age=20, name="John")
```

Output:

John 20

3. Default Arguments

- Parameters have **default values**
- If no value is given, default is used

Example:

```
def greet(name="John"):
    print("Hello", name)
```

```
greet()      # uses default
greet("Ali") # overrides default
```

Output:

```
Hello John
Hello Ali
```

3. Scope of variables in Python.

➔ **Scope** refers to the **region where a variable can be accessed** in a program.

◆ Types of Variable Scope

1. Local Scope

- Variable declared **inside a function**
- Can be used **only within that function**

Example:

```
def show():
    x = 10 # local variable
    print(x)
```

```
show()
```

2. Global Scope

- Variable declared **outside all functions**
- Can be used **anywhere in the program**

Example:

```
x = 20 # global variable
```

```
def show():  
    print(x)
```

```
show()  
print(x)
```

3. Using global Keyword

- Used to **modify global variable inside a function**

Example:

```
x = 10
```

```
def change():  
    global x  
    x = 50
```

```
change()  
print(x)
```

Output:

```
50
```

4. Built-in methods for strings, lists, etc.



1. String Methods

Method	Description
upper()	Converts to uppercase
lower()	Converts to lowercase
strip()	Removes spaces
replace()	Replaces text
split()	Splits string into list
find()	Finds position

Example:

```
text = " hello python "
```

```
print(text.upper())    # HELLO PYTHON
```

```
print(text.strip())    # hello python
```

```
print(text.replace("python", "Java"))
```

◆ 2. List Methods

Method	Description
append()	Adds element at end
insert()	Adds at specific position
remove()	Removes element
pop()	Removes last element

Method	Description
--------	-------------

sort()	Sorts list
--------	------------

reverse()	Reverses list
-----------	---------------

Example:

```
numbers = [3, 1, 2]
```

```
numbers.append(4)
```

```
numbers.sort()
```

```
print(numbers) # [1, 2, 3, 4]
```

10. ADVANCED PYTHON [MAP(),REDUCE(),FILTER()]

THEORY :

1. How functional programming works in Python.



1. What is Functional Programming?

- Functional programming is a style of programming where:
 - Functions are treated as **first-class objects**
 - Focus is on **using functions to perform operations**
- It avoids changing data and uses **pure functions**

2. Key Features of Functional Programming

◆ 1. First-Class Functions

- Functions can be:
 - Assigned to variables
 - Passed as arguments
 - Returned from other functions

Example:

```
def greet():  
    return "Hello"
```

```
x = greet  
print(x())
```

◆ 2. Lambda Functions (Anonymous Functions)

- Small functions using lambda

```
square = lambda x: x * x  
print(square(4)) # 16
```

◆ 3. Higher-Order Functions

- Functions that take other functions as arguments

Common Functional Tools

- i. `map()`
- ii. `reduce()`
- iii. `filter()`

2. Using `map()`, `reduce()`, and `filter()` functions for processing data.



1. `map()`

- Applies a function to all elements

```
numbers = [1, 2, 3]
```

```
result = list(map(lambda x: x * 2, numbers))  
print(result)
```

Output:

```
[2, 4, 6]
```

2. `filter()`

- Filters elements based on condition

```
numbers = [1, 2, 3, 4]
```

```
result = list(filter(lambda x: x % 2 == 0, numbers))  
print(result)
```

Output:

```
[2, 4]
```

3. reduce() (from functools)

- Reduces list to a single value

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(lambda a, b: a + b, numbers)  
print(result)
```

Output:

10